Relatório de Testes — Gilded Rose Refactoring Kata (TypeScript)

Integrantes

Nome	RA
Julia Victoria da Silva	1110482322014
Nicolas Domingos da Silva	1110482322026

1. Link do projeto analisado

🔗 **https://github.com/NicolasDomingos09/GildedRose-Refactoring-Kata**

- Fork público do repositório `emilybache/GildedRose-Refactoring-Kata`.
- Pasta analisada: `TypeScript/`

O que o código faz: gerencia o estoque da taverna "A Rosa Dourada" de um RPG. A cada fim de dia, `GildedRose.updateQuality()` atualiza cada item:

Item	Antes do prazo	Após expirar	Restrições
Item comum	Qualidade −1	Qualidade **−2**	Nunca negativa
`Aged Brie`	Qualidade +1	+1	Teto 50
`Sulfuras, Hand of Ragnaros`	Imutável	Imutável	Qualidade 80
`Backstage passes`	+1 (>10 dias), +2 (≤10), +3 (≤5)	**Zera** após o show	Teto 50
`Conjured` *(regra nova)*	Qualidade −2	Qualidade −4	Nunca negativa

2. Classes testadas

Classe / membros	Papel	Estratégia
`Item`	Entidade: `name`, `sellIn`, `quality`	Testada indiretamente via `GildedRose`
`GildedRose.updateQuality()`	Regra central de envelhecimento do estoque	**Testes de caracterização** (congelam o comportamento legado) + testes da regra nova

Classe / membros	Papel	Estratégia
`GildedRose` (snapshot de aprovação)	Execução de 30 dias simulados	Golden master (snapshot Jest)

Cobertura final alcançada: **100% de linhas e declarações** em `app/gilded-rose.ts` (96,87% de ramos — único ramo não alcançável sem mutating test: qualidade de item lendário abaixo de 50).

---

3. Testes unitários realizados

Suite principal: `test/jest/gilded-rose.spec.ts` — **22 testes**, organizados em TDD:

Fase 1 — Caracterização da lógica legada (17 testes)

#	Cenário	Resultado esperado
1	Item comum, 1 dia	`sellIn` −1, `quality` −1
2	Item comum expirado	`quality` −2/dia
3	Item comum com quality 0	Nunca fica negativo
4	Item comum expirado vários dias	Piso em 0
5–7	`Aged Brie` recém, expirado, teto	+1/dia; duplo incremento expirado; teto 50
8–9	`Sulfuras` normal e expirado	Imutável
10	Passes >10 dias	+1/dia
11	Passes ≤10 dias	+2/dia
12	Passes ≤5 dias	+3/dia
13	Passes com quality 49	Teto 50
14	Passes com show realizado	Quality **0**
15	Vários itens numa chamada	Cada item segue sua regra

Fase 2 — TDD da regra nova `Conjured` (5 testes, escritos ANTES da implementação)

#	Cenário	Resultado esperado
16	`Conjured` com prazo válido	`quality` −2/dia
17	`Conjured` expirado	`quality` −4/dia
18–20	Piso em 0 (3 variantes)	Nunca fica negativo

Além dos 22, a suíte `approvals.spec.ts` valida o snapshot do golden master de 30 dias (regenerado após a mudança intencional de comportamento).

---

4. Nova regra adicionada: itens `Conjured`

Regra de negócio implementada (a canônica da kata): itens mágicos (nome iniciado com `Conjured` — o exemplo do estoque é o `Conjured Mana Cake`) **degradam a qualidade duas vezes mais rápido que itens comuns**:

Situação	Item comum	Item `Conjured`
Prazo de venda válido (`sellIn` ≥ 0)	Quality −1/dia	**Quality −2/dia**
Prazo expirado (`sellIn` < 0)	Quality −2/dia	**Quality −4/dia**
Em qualquer situação	Piso 0	**Piso 0 (nunca fica negativo)**

Onde vive a regra (`app/gilded-rose.ts`). Os dois pontos de degradação do legado passaram a chamar um helper único:

```
private isConjured(item: Item): boolean {
  return item.name.startsWith('Conjured');
}

// Itens "Conjured" (mágicos) degradam a qualidade duas vezes mais rápido
// que itens comuns: -2 por dia, e após a expiração os dois decrementos
// somam -4 por dia. A qualidade nunca fica negativa.
private decreaseQuality(item: Item): void {
  if (this.isConjured(item)) {
    item.quality = item.quality - 2;
    if (item.quality < 0) {
      item.quality = 0;
    }
  } else {
    item.quality = item.quality - 1;
  }
}
```

Ordem TDD respeitada: (1) testes da lógica antiga escritos primeiro e aprovados contra o código intocado → (2) testes da regra `Conjured` escritos e **falhando** (fase vermelha: o legado tratava Conjured como item comum: −1/dia / −2/dia) → (3) só então a função foi implementada para satisfazer os testes (fase verde: 22/22 ✅). A rede de segurança garantiu que nenhum item preexistente mudou de comportamento — o diff do golden master confirmou que **somente as linhas de `Conjured Mana Cake` mudaram**.

Execução em runtime do artefato compilado (`app/gilded-rose.js`, gerado pelo `tsc`) assumindo 3 dias com o estoque padrão — `Conjured Mana Cake` degrada 6 → 4 → 2 → 0:

```
-------- day 0 --------            -------- day 2 --------
name, sellIn, quality              name, sellIn, quality
+5 Dexterity Vest, 10, 20          +5 Dexterity Vest, 8, 18
Aged Brie, 2, 0                    Aged Brie, 0, 2
Conjured Mana Cake, 3, 6           Conjured Mana Cake, 1, 2
Sulfuras, Hand of Ragnaros, 0, 80  Sulfuras, Hand of Ragnaros, 0, 80

-------- day 1 --------            -------- day 3 --------
```

```
name, sellIn, quality              name, sellIn, quality
+5 Dexterity Vest, 9, 19           +5 Dexterity Vest, 7, 17
Aged Brie, 1, 1                    Aged Brie, -1, 4
Conjured Mana Cake, 2, 4           Conjured Mana Cake, 0, 0   ← -2/dia, piso em 0
Sulfuras, Hand of Ragnaros, 0, 80  Sulfuras, Hand of Ragnaros, 0, 80
```

---

5. Resultado das compilações e dos testes

- ✅ `app/gilded-rose.ts` **e** `test/**` **compilam sem erros**. O artefato emitido foi executado em runtime com sucesso (seção 5).

5.1 Testes unitários (Jest)

```
$ npx jest

PASS test/jest/gilded-rose.spec.ts
PASS test/jest/approvals.spec.ts
-----------------------------|---------|----------|---------|---------|
| File                       | % Stmts   | % Branch   | % Funcs   | % Lines   |
| -------------------------- | --------- | ---------- | --------- | --------- |
| All files                  | 100       | 97.05      | 100       | 100       |
| app                        | 100       | 96.87      | 100       | 100       |
| gilded-rose.ts             | 100       | 96.87      | 100       | 100       |
| test                       | 100       | 100        | 100       | 100       |
| golden-master-text...      | 100       | 100        | 100       | 100       |
| -------------------------- | --------- | ---------- | --------- | --------- |

Test Suites: 2 passed, 2 total
Tests:       22 passed, 22 total
Snapshots:   2 passed, 2 total
Time:        5.084 s
Ran all test suites.
```

5.2 Evidência do ciclo TDD (fase vermelha)

Antes de implementar a regra `Conjured`, os testes falharam como esperado (redu → green):

```
● ...› Itens Conjured (nova regra de negócio) › degrada quality em 2 por dia
  Expected: 4   Received: 5        ← comportamento atual tratava Conjured como
item comum
● ...› degrada quality em 4 por dia após o prazo de venda expirar
  Expected: 2   Received: 4

Tests: 2 failed, 20 passed, 22 total     ← RED (correto)
```

Após implementar `decreaseQuality()`/`isConjured()` em `app/gilded-rose.ts`:

```
Test Suites: 2 passed, 2 total
Tests:       22 passed, 22 total        ← GREEN
```

O snapshot de aprovação foi regenerado (`npx jest -u`) e o diff confirmou que **apenas as linhas de `Conjured Mana Cake` mudaram** — a regra nova foi isolada sem efeito colateral nos itens legados.

---

6. Conclusão para análise e refatoração

O que a adição de testes revelou:

1. **A rede de segurança funcionou.** Os 17 testes de caracterização congelaram o comportamento legado (inclusive quirks como o duplo incremento de `Aged Brie` expirado e o zrateamento de passes antes do `sellIn` ser decrementado). Depois disso foi possível mexer no código com confiança.
2. **O código legado é um monolito de `if/else` aninhados.** A regra nova precisou tocar 3 pontos de degradação. Viabilizei a mudança de forma mínima (`isConjured()` + `decreaseQuality()` com piso em 0), mas o passo seguinte natural é o **refactoring canônico desta kata**: extrair um método por tipo de item (polimorfismo — `ItemUpdater`/`ConjuredUpdateStrategy`), eliminando o encadeamento de condições e tornando cada regra um método pequeno e nomeado.
3. **Dividas técnicas do tooling identificadas na análise:**
   - `rimraf app/**/*.js` quebra no Windows (rimraf v4); corrigir para `rimraf "app/**" "test/**"` ou usar `clean` via Node;
   - TypeScript 4.9 fixado conflita com tipos de dependências novas; atualizar `typescript` para ≥5.x;
   - Sem um CI — adicionar GitHub Actions rodando `npm run test:jest` garantiria que toda a equipe mantém essa cobertura.

Veredito final: compilação limpa no código do projeto, **22/22 testes passando**, **100% de cobertura de linhas**, regra nova entregue via ciclo TDD completo (testes → red → implementação → green) com evidência de isolamento pelas captures de golden master. O projeto está em condição de receber a refatoração estrutural com risco baixo.